STUDENT EDITION · PYTHON, LOGIC, AND BEGINNER MATHEMATICS

MEDNEX

A complete, step-by-step demonstration
for curious beginners

70-page learning guide
Understand the app from the first click to the final response. Learn the beginner Python ideas behind it. Follow the toy probability calculations one small step at a time. No advanced mathematics is required.

A careful note before we begin

MEDNEX is a local classroom software demonstration, not a medical service. Use only fictional examples. Its sample data and labels are synthetic. Nothing in the application diagnoses, triages, or recommends treatment.

A friendly map

This guide is designed for a student who is new to coding and simple statistics. Read in order, or use the page map to find a topic. Every formula is translated into ordinary words before it is used. Code examples are small pieces for learning; they are not a replacement for the real project files.

Pages	Learning path
3–9	What MEDNEX is, its two modes, and the complete route from browser to reply.
10–15	Python basics: values, variables, strings, lists, dictionaries, conditions, loops, and functions.
16–23	Browser/server basics, templates, and JSON.
24–31	Pattern matching, parsing, and structured facts.
32–41	Safety gates, validation, graph ideas, symbolic rules, agent cycles, and follow-ups.
42–49	Probability foundations and a worked Bayes example using the synthetic data.
50–57	Machine learning concepts and beginner explanations of evaluation measures.
58–65	Entropy and information gain, explained step by step with actual example parameters.
66–70	Outputs, tests, limitations, a complete classroom demonstration, and review.

Reading tip: When you see a value like 0.8, read it as “eight tenths,” or 80 out of 100. When you see a code name, it is simply a label the programmer chose. You can skip the code and still understand the story.

The implementation details were checked against this repository's Python files and bundled data. The guide explains the system as it is implemented; it does not claim that these steps are the historical order in which the project was built.

01

What is MEDNEX?

MEDNEX is a small local web application. A person types a message into a browser page. Python receives it, follows fixed rules, and sends a response back. This project is interesting because it looks like a chat, but it does not use a large language model to invent its answer.

Instead, the program recognizes some known phrases, chooses prepared text from local files, and puts that text into a sentence structure written by a developer. In a separate mode, it runs educational examples of logic, probability, machine learning, and question selection.

> **Big idea:** MEDNEX is like a carefully labeled card catalogue. It can find cards it knows about and arrange them in a helpful order. It does not understand every possible sentence.

Why build it? To demonstrate that a predictable, inspectable program can power a conversational interface. You can trace a response back to code and data. The trade-off is limited coverage: unfamiliar language may not match, and a matching phrase does not mean the program understands the person.

Key words: “local” means the server runs on this computer; “deterministic” means the same supported input and same data normally lead to the same output; “LLM” means large language model.

Two modes, two jobs

The page offers two different experiences. They may look similar, but their internal work is different. Keeping this distinction clear is one of the most important things to understand about MEDNEX.

General Information	Reasoning Agent
Matches supported message wording to a guidance topic.	Accepts a clearly fictional or synthetic case.
Returns prepared educational text.	Parses facts and runs separate teaching engines.
Does not run Bayesian, graph, symbolic, or ML reasoning.	Runs those engines, then hides technical output in chat.
May ask for clarification when it cannot match.	May ask a planner-selected educational follow-up.

Why two modes? A general chat response and a reasoning experiment solve different software problems. A retrieval-and-template path is straightforward and controlled. The agent mode lets a class explore how structured data and toy reasoning methods connect.

> **Do not mix the claims:** The chat does not use a language model. The reasoning agent's synthetic probability numbers do not create the chat's wording. Its final user-facing answer is also built from fixed template code.

When explaining MEDNEX aloud, say which mode you are showing before entering an example. That prevents a viewer from thinking every answer has passed through every engine.

What problem does it solve?

A web page needs a way to receive text, understand which supported topic was mentioned, choose a response, and display it. MEDNEX demonstrates this whole path using ordinary software pieces rather than an AI text generator.

1. **Receive:** get the message from the web page.
2. **Check:** reject bad or out-of-scope requests.
3. **Recognize:** find known terms with explicit patterns.
4. **Represent:** store the result in named fields.
5. **Respond:** select approved content and arrange it.
6. **Display:** send the result back to the browser.

For the agent, additional educational steps calculate graph associations, rule traces, synthetic probabilities, model predictions, and candidate questions. These are separate examples inside the same application.

Why is predictability useful? A student can inspect the response path. A developer can write a test that says “when this supported phrase is sent, this status and these fields are returned.” This is easier to trace than text that could be generated differently each time.

What is not solved? MEDNEX does not understand arbitrary language, know personal context, or establish that a synthetic pattern applies to reality. It is an engineering demonstration, not a medical decision tool.

The whole route at a glance

```
Browser page
   ↓ sends a JSON message
Python local web server
   ↓ checks route and request
General chat OR Reasoning Agent
   ↓ creates a structured result
Fixed response builder
   ↓ returns JSON
Browser displays text
```

Think of a restaurant. The browser is the customer placing an order. The server is the counter receiving it. The selected mode is the kitchen station. The structured response is the tray with labeled sections. The browser is the person carrying it back to the table.

Each boundary has a job. The browser should not decide probability calculations. The parser should not decide how a button looks. The response builder should not pretend it can infer facts that were not extracted.

> **Vocabulary:** A *request* travels from browser to server. A *response* travels back. An *endpoint* is a named server route such as `/api/chat`. JSON is a text format for labeled data.

Later pages follow one fictional case through each step. The same case is used repeatedly so you can see which facts are carried forward and which numbers belong only to a particular exercise.

What “no LLM” means

An LLM can produce new text by predicting language patterns. MEDNEX's chat follows a different method: it uses written patterns to recognize a limited vocabulary, then fills written sentence frames with known labels and content.

Fixed template

“I can share general information about [topic].” The bracket is filled with a recognized topic.

Prepared data

A local file provides a label and prewritten comfort and safety wording for each supported topic.

The combination can feel conversational, but the grammar is not learned dynamically. If the code has not been written to handle a phrase or a combination, it may respond awkwardly or ask for clarification.

Why demonstrate this? It makes the source of each sentence inspectable and repeatable. It also makes the limits visible: the system only knows what the developers added to its pattern lists, templates, and data files.

“No LLM” does not mean “no algorithms.” The agent includes classical algorithms: graph traversal, if/then rules, probability arithmetic, and standard machine-learning models. They calculate or select; they do not write the answer prose.

What counts as structured data?

A sentence is flexible: “A fictional person has a cough for three days.” A program is easier to reason about when it turns that sentence into labeled pieces, such as a symptom name, a present/absent marker, and a duration.

Field name	Example value	Plain meaning
`name`	`cough`	Which supported symptom phrase matched
`is_negated`	`false`	The sentence did not explicitly deny it
`duration`	3 days	A duration was stated
`source_text`	“cough”	Text that supports the extracted item

This is similar to filling in a form. Each box has one meaning. A later step can check whether a value is missing without rereading the whole sentence.

> A field being absent from the form means “not supplied or not extracted.” It does not mean the real-world fact is false.

Why structure? Typed fields support validation, calculations, and tests. They also make uncertainty easier to preserve: ambiguity and contradiction can be recorded explicitly instead of hidden in a summary.

Prepare the classroom demo

MEDNEX is run from the project folder. Use a fictional demonstration input only; do not enter a real person's information.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python app.py
```

The first command creates a separate Python environment for project packages. The second activates it in PowerShell. The third installs the package versions listed by the project. The fourth starts the local server.

Open the loopback address printed in the terminal, usually `http://127.0.0.1:8000`. “Loopback” means the address points back to this computer. Stop the process with Ctrl+C.

> **Before class:** install dependencies once, start the server, wait for the local URL, and open the page. If the page cannot load, check the terminal for the server message before changing code.

Why use a virtual environment? It keeps this project's packages separate from other Python projects. The application uses Python, Pydantic, NetworkX, scikit-learn, Matplotlib, and pytest. The web page uses ordinary HTML, CSS, and JavaScript.

Instructions, print, and variables

Python is a programming language: a way to write instructions a computer can run. MEDNEX uses it for the local server, parsing, data checks, calculations, and tests. Read code one line at a time. Ask what value is here before a line, what it does, and what value exists after.

A Python program is a sequence of instructions. Python reads and runs them. `print` asks Python to display something in the terminal.

```
print("Hello, MEDNEX!")
print(2 + 3)
```

The first line displays words. The quotation marks show Python that the content is text. The second line calculates a sum and displays the result, `5`.

Why begin here? A visible result helps a new programmer confirm that a program ran and understand the order of instructions. The real MEDNEX server prints its local web address when it starts.

> **Try it mentally:** If you put `print("Start")` and then `print("Finish")`, what appears first? The first instruction runs first: Start, then Finish.

Words inside quotes are a **string**. A number like `5` without quotes is numeric data. The computer can add numeric values, while quoted text is treated as characters.

In Python, capitalization matters. `print` works; `Print` is a different name and normally causes an error.

A variable is a name connected to a value. Imagine a labeled box: the label helps you use the value later.

```
topic = "cough"
days = 3
print(topic)
print(days)
```

Here `topic` points to text and `days` points to a number. The equals sign in an assignment means “store this value under this name.” It is not the same as a math equation asserting two sides are always equal.

MEDNEX uses names such as `message`, `extraction`, and `comfort_steps`. Meaningful names make code easier to read, especially when one value is passed between several functions.

> **Good naming:** `question_count = 2` tells a reader more than `x = 2`. Python names are case-sensitive: `topic` and `Topic` are different names.

Variables can be updated. If `days = 3` is followed by `days = days + 1`, Python calculates the right side (4) and stores it under `days`. Capitalization matters: `print` and `Print` are different names.

Python uses indentation to show which lines belong inside a function, condition, or loop. A colon commonly begins a block.

Numbers, percentages, and text strings

Python can do arithmetic. A decimal such as 0.8 is eight tenths. To turn it into a percentage, multiply by 100 and add the percent sign: **0.8 × 100 = 80%**.

Decimal	Fraction	Percent
0.5	5 out of 10	50%
0.25	25 out of 100	25%
0.8	8 out of 10	80%

When probabilities describe all possible outcomes, their total should be 1, or 100%. For example, 0.4 + 0.35 + 0.25 = 1. Those are the three starting values in MEDNEX's synthetic Bayesian example.

Multiplication: “0.4 times 0.8” can be read as taking 80% of 0.4. The result is 0.32. **Division:** “0.24 divided by 0.3” asks how many parts of size 0.3 fit into 0.24; the result is 0.8.

> These conversions explain the arithmetic only. A project number marked as a probability is not automatically a trustworthy real-world probability.

A string stores text. MEDNEX receives a user's message as a string, checks it, and sometimes combines pieces into a response.

```
name = "cough"
sentence = "I can share information about " + name + "."
print(sentence)
```

The plus sign between strings joins them; it does not add their meaning. The output is a longer string. MEDNEX also uses formatted strings, which put a named value inside a sentence.

```
topic = "cough"
sentence = f"General information about {topic}."
print(sentence)
```

The letter `f` before the opening quote tells Python to replace the braces with the value of the variable. This is one way deterministic templates fill in a topic name.

Why not ask Python to write the rest? Python is following a sentence the developer already wrote. It is not generating a new paragraph based on broad language understanding.

Careful: Punctuation and grammar must make sense around the inserted value. A template that says “a [symptom]” may sound wrong when the label starts with a vowel. Template design needs examples and tests.

Lists and dictionaries

A list holds several values in a particular order. MEDNEX uses lists for symptoms, comfort steps, and questions.

```
symptoms = ["cough", "fever"]
print(symptoms[0])
print(len(symptoms))
```

The first item is at position 0, not 1. This is called zero-based indexing. The code displays `cough`, then `2`. The built-in `len` function counts the items.

A list can be empty, which means there are no recorded items. That is different from a list containing the word “absent.” The list `[]` says there is no entry; `["absent"]` contains an explicit value.

In MEDNEX, a list of extracted facts can be looped over. Another part of the code may then look at each item and decide whether it is present, negated, ambiguous, or conflicting.

Mini-check: In `steps = ["rest", "water", "ask"]`, what is `steps[1]`? It is `"water"`, the second item.

A dictionary stores values under keys. It is like a collection of named drawers: ask for one key and get its value.

```
topic = {
    "label": "cough",
    "comfort_steps": ["rest", "drink fluids"]
}
print(topic["label"])
```

The dictionary has keys `label` and `comfort_steps`. The first maps to a string; the second maps to a list. JSON files use a similar key-and-value structure, which is why they work well for MEDNEX's curated topics and parameters.

A dictionary does not mean the program understands the subject. It means information has been stored under a predictable name. Code still needs to decide what those names mean and how to use them.

For example, the general guidance file has a topic identifier, a display label, and lists of prepared content. The response function retrieves the topic and uses its fields.

Beginner distinction: A list is useful when order matters; a dictionary is useful when you want to retrieve a value by a meaningful key.

Booleans, if-statements, and loops

A Boolean is either `True` or `False`. A program uses an `if` statement to choose a path based on a condition.

```
is_negated = False
if is_negated:
    print("The symptom was denied.")
else:
    print("The symptom was not marked as denied.")
```

Because `is_negated` is false, the `else` branch runs. The indented lines belong to their branch. The colon and indentation are part of Python's syntax.

MEDNEX uses conditions for important gates. For example, if safety says the simulation must stop, it returns a boundary response. If extraction is contradictory, the agent avoids making a clean summary.

Why use a Boolean? A clearly named yes/no value is easier to validate than a sentence such as “maybe the person perhaps does not have this.” When a fact is not clear, the parser should record uncertainty rather than force True or False.

A loop repeats instructions for each item in a collection. This avoids copying the same lines again and again.

```
steps = ["rest", "drink fluids"]
for step in steps:
    print("- " + step)
```

First, `step` becomes `"rest"` and Python prints it. Next, `step` becomes `"drink fluids"` and Python prints that. When the list ends, the loop ends.

The general chat loops through matching guidance topics to gather their comfort information. The report formatter loops through facts to make readable lines.

A loop has a start, a repeated body, and a stopping point. A bug in the stopping point can repeat forever, so programmers test loops with small examples.

> **Try it:** What happens if `steps = []`? The loop body runs zero times because there are no items. No bullet is printed.

Functions: named mini-programs

A function is a named set of instructions. It can accept inputs, do work, and return a result.

```
def greeting(topic):
    return f"General information about {topic}."

answer = greeting("cough")
print(answer)
```

`def` begins a function definition. `topic` is an input parameter. `return` sends a value back to the code that called the function. Calling `greeting("cough")` runs the body using that input.

MEDNEX functions have focused jobs. A parser extracts facts. A guidance function builds a reply. A validator checks a case. Small jobs are easier to explain and test than one giant function that does everything.

Functions can call other functions. That is how a route can check a request, call a chat function, then serialize a result.

Types, models, and errors

A type describes what kind of value is expected: text, number, Boolean, list, or a structured model. Python allows flexible values, but MEDNEX uses type hints and Pydantic models to make expectations clearer.

```
def add_days(days: int) -> int:
    return days + 1
```

`int` means whole-number input; the arrow says the function should return a whole number. A type hint is a signpost for readers and checking tools. It does not by itself make every input safe.

Pydantic models add runtime checks. For example, a response model can require a status to be one of a small set of allowed words and require fields to have the expected types.

Why types? They make data contracts visible. If a response needs a list of comfort steps, the data model can say it is a list of strings. The model can reject an unexpected shape instead of letting the error travel farther.

> Type-valid data is not automatically true, useful, or safe. A perfectly valid number can still be an invented teaching number.

An error message tells you that the program could not complete an instruction. Errors are not proof that you are bad at programming. They are information about what the computer could not do.

Error type	Simple example	Likely clue
Syntax error	Missing closing quote	Python could not read the instruction
Type error	Add text to a number	These value types do not fit the operation
Value error	Convert “hello” to a number	The type exists, but this value is unacceptable
File error	Open a missing data file	The path or file is unavailable

MEDNEX validates JSON data as it loads. If required fields or values are wrong, validation raises an explicit error. That is preferable to silently pretending the data loaded correctly.

Beginner debugging method: read the last line of the error first, find the file and line mentioned, inspect the nearby code, and change one thing at a time. Then rerun the smallest relevant test.

The browser's job

The browser shows the interface: the title, mode buttons, message area, and text box. JavaScript reads the user's message and sends it to the selected endpoint.

```
POST /api/chat
Content-Type: application/json

{"message": "Please give general information about cough."}
```

`POST` means the browser is sending data to the server. `Content-Type` says what format is in the request body. The JSON object uses a key called `message` and a text value.

The browser is not the part that decides which topic to select. It mostly sends the input and displays the returned response text. Keeping the interface separate from the reasoning code makes each part easier to change.

When the server returns an error, the page can show an error state. When the server is working, the page may show a temporary loading indicator.

The server's job

The Python server listens for local requests. It serves the page files and handles a small set of approved routes. For the General Information mode, the route is `/api/chat`.

Before calling response logic, the server checks that the route exists, the body size is allowed, the request says it contains JSON, and the decoded data has the expected shape.

Why do these checks matter? A function should not have to guess whether the incoming value is text, a number, a missing object, or an enormous request. Early checks make failures clearer and limit the work the server needs to do.

> **Request → function → response:** The server receives a dictionary-like JSON object, checks that `message` is text, and calls the chat function with that text.

The server binds to `127.0.0.1`, the loopback address. This demo is designed for local classroom use, not public internet deployment.

JSON is a labeled message

JSON stands for JavaScript Object Notation. Despite its name, Python can read and write it too. It is text with familiar structures such as objects, lists, numbers, and strings.

```
{
  "status": "needs_clarification",
  "response": "Please describe a supported topic.",
  "understood": []
}
```

The braces make an object. Each key is a label; each value is the data for that label. A list uses square brackets. Double quotes mark text.

JSON gives the browser a predictable response shape. The browser can read `response` and display it, while tests can inspect `status` or `understood`.

A JSON response is structured, even if one of its fields contains a paragraph. Structure is about named fields; grammar is about how words fit together. MEDNEX has structured fields and fixed prose templates, not an automatic grammar or language generator.

Templates create the sentences

A template is a sentence frame written before the user sends a message. The program fills a placeholder with a selected label or content string.

```
topic_label = "cough"
opening = f"General information about {topic_label}."

# The result is assembled, not newly authored:
# "General information about cough."
```

Real MEDNEX code has branches for safety, negation, clarification, and matching topics. It also joins prewritten paragraphs and list items. Those branches decide which prepared text is appropriate to show.

Why templates? They are easy to review, test, and keep consistent. They also make the response vocabulary limited. If labels do not fit the sentence frame, grammar may sound awkward. That is a template design challenge, not evidence that a hidden model wrote the answer.

In the Reasoning Agent, the user's actual parsed symptom names and guidance library are used. Synthetic probability values do not appear in the plain-English answer.

Prepared guidance data

The file `general_guidance.json` stores a small collection of topics. A topic has an identifier and label plus prepared lists for general comfort, when to seek professional help, and selected urgent signs.

Why store text in data instead of burying it in code? A clearly structured data file makes the content easier to review and update. The code can use the same assembly steps for multiple topics.

There are ten general guidance topics. The parser can recognize more symptom words than the guidance file supports. Recognition does not guarantee that a guidance entry exists.

> The guidance is an educational example, not a clinical guideline. The project does not claim that the list is comprehensive or personalized.

The topic has aliases in its JSON entry, but the current chat selects topics by the canonical name returned by the parser. The parser's actual aliases are defined separately in `nlp/patterns.py`.

Structured response versus grammar

Imagine a response object with fields for status, response text, topics understood, and lists of guidance. The fields are structured. The sentence inside `response` is made from written text.

Structure asks:
Which field stores the response? What status was chosen? Which topic was matched?

Grammar asks:
Does the sentence sound natural? Do subjects and verbs fit together? Is punctuation correct?

MEDNEX addresses structure with Python models and JSON. It addresses grammar with prewritten sentence frames and labels. It does not use a natural-language grammar engine to invent a sentence for every possible input.

A reader should therefore expect predictable sections, not a flexible conversation. A response can be structured correctly yet sound slightly unnatural. Tests can check exact wording for known cases, but good prose still needs human review.

Try the General Information chat

Use a general, non-personal request such as:

```
Please give general information about cough.
```

Observe that the response selects a supported topic, displays general educational content, and does not provide a diagnosis, medicine, or dose. Then try a supported combination such as “Tell me general information about cough and fever.” The exact phrasing must match what the parser knows.

Try a question about a topic outside the supported list. The system may ask for clarification rather than invent content. Try an explicitly negated phrase; the parser may retain it as not present.

Do not use “I have a cough” in this demo. The safety checks recognize some personal-health language and stop rather than provide individualized care. The system is not designed to assess that message.

Classroom observation checklist: Which route was used? What topic was recognized? What is fixed wording? What was returned as data? What happened when the wording was not supported?

When wording is not recognized

Pattern-based programs need specific words and phrases. An alias is an extra phrase that the program maps to the same canonical name. For example, several everyday phrases are mapped to a single symptom name.

MEDNEX includes a finite set of aliases and a small set of spelling variants. It does not contain a general spell checker. A typo not in the list can fail to match.

Why ask for clarification? If the code cannot match a supported topic, it is safer and more honest to say so than to pretend it knows which content applies. In software terms, “I do not know” is a valid result.

Different paths have different fallback behavior. The general chat can ask for a supported topic. The agent builder can refuse a summary for unsupported or ambiguous content. The reasoning planner can stop uncertain.

> A parser match is not proof of correct understanding. The word “cold,” for example, can have meanings beyond the exact topic the demo intended. Context handling is limited.

What is a pattern?

A pattern describes text the program knows how to find. A regular expression is a compact pattern language. For example, a pattern can look for a number followed by a time unit such as “days” or “hours.”

```
"cough"       → canonical symptom: cough
"three days"  → amount: 3, unit: days
"no fever"    → symptom: fever, marked negated
```

MEDNEX's parser stores symptom aliases and measurement patterns in `nlp/patterns.py`. It normalizes some forms, uses case-insensitive matches, and then creates structured models.

Why patterns? They are understandable, testable, and predictable for the cases they cover. Their limitation is the same: if the words differ from the known pattern, the program may not find them.

Matching a word is a technical event. Correctly understanding its intent requires context. The parser uses only limited rules for context, so its confidence should never be overstated.

Normalize before matching

Normalization makes a few known variations more consistent. MEDNEX lowercases text and replaces some punctuation forms. It also maps certain aliases, such as a supported alternate measurement label, to a canonical word.

For example, the normalizer may turn uppercase letters into lowercase so the phrase “COUGH” can be matched by a case-insensitive pattern just like “cough.” It also includes a small number of phrase substitutions.

Why normalize? Without it, the same word typed with different capitalization or a known alias could require duplicate rules. Normalizing reduces unnecessary variation.

> Normalization is limited. It does not understand synonyms in general, repair arbitrary spelling, or determine the meaning of a whole paragraph. It can also change context if rules are too broad.

After normalization, the parser uses specific patterns to find supported symptoms, durations, and measurements. It preserves source text for extracted observations so a reader can see which part of the input produced a value.

Reading a regular expression gently

A regular expression (often called regex) is a tiny pattern language. The following example is a simplified illustration, not the exact full MEDNEX pattern:

```
\b(cough|coughing)\b
```

- `\b` asks for a word boundary.
- `( ... )` groups alternatives.
- `|` means “or.”
- `cough` and `coughing` are the accepted words.
- The final `\b` helps avoid matching those letters inside a longer unrelated word.

The real code safely escapes the known aliases before joining them into patterns. It sorts longer aliases first so an overlapping long phrase can be matched before its shorter part.

Regex is useful for finite patterns, such as “a number followed by bpm.” It is not a substitute for understanding meaning. “No cough” and “cough” share the same symptom word; a separate negation rule must examine nearby text.

Numbers and units

The parser can recognize selected formats for age, duration, temperature, heart rate, breathing rate, oxygen saturation, and blood pressure. It also converts some number words to numeric values: for instance, “three” can become 3.

Why require units and labels? The number 96 by itself is ambiguous. A recognized phrase like “oxygen saturation is 96 percent” gives the parser more clues about which field to fill.

MEDNEX has typed models that validate basic shapes. For example, oxygen saturation is constrained to a range from 0 to 100. Blood-pressure fields must be supplied as a pair.

> A value passing a data-type check does not mean it is medically plausible, measured correctly, or interpreted safely. This parser's extraction is an educational software example.

Each extracted observation records its name, value, source text, and source type. This lets later code distinguish a value parsed from a sentence from a value supplied as structured data.

Negation: “no” changes the meaning

Negation means denying a statement. “Cough” and “no cough” contain the same topic word but communicate different states. MEDNEX searches a nearby part of the sentence for known negation phrases and stores a Boolean marker.

```
"cough"     → name="cough", is_negated=False
"no cough"  → name="cough", is_negated=True
```

Why store both the name and the marker? Different parts of the program need to know both what topic was mentioned and whether the text said it was absent.

The parser also handles some contrast words such as “but” to limit the scope of negation. Still, this is a simple window-based method. Complex sentences can be misunderstood.

> **Test idea:** Compare “no fever, but cough” with “fever and cough.” Verify that the program does not apply the first negation to both topics.

If both present and absent versions of a symptom appear, the system can record a contradiction. It should not silently select whichever version is more convenient.

Ambiguous and unsupported text

Some wording is unclear: “maybe a cough” is not the same as a direct report. Some wording may also contain concepts outside the parser's supported vocabulary. MEDNEX records selected ambiguous and unsupported statements so downstream steps can avoid treating them as certain facts.

Why keep a list of uncertainty? If a program discards hard-to-parse content, a later component might mistake the remaining small set of facts for a complete story. Recording what could not be understood makes the gap visible.

The parser has an extraction clarity indicator from 0 to 1. This number describes the clarity of deterministic text extraction under its rules. It is not a probability that the extracted facts are medically correct.

> Do not translate “90% extraction clarity” into “90% sure of the health result.” Those are completely different claims.

Unsupported or ambiguous agent inputs lead to an uncertain result or a response asking for a clearer fictional description. The program does not have to force every sentence into a known field.

Contradictions are not averaged away

A contradiction occurs when the same fact is represented in opposing ways—for example, a symptom is both present and absent. The parser compares symptom states and selected observation values to find some such conflicts.

Why not choose one statement? MEDNEX has no reliable basis for deciding which one is correct. Picking one silently would hide a data-quality problem. The safer software behavior is to surface the conflict and stop the educational reasoning path as uncertain.

```
Input: "A fictional person has fever but has no fever."
Result: contradiction recorded
Planner: UNCERTAIN
User response: asks to correct the fictional description
```

This is a data consistency decision, not a clinical judgment. The contradiction says the text cannot be interpreted unambiguously under the program's supported rules.

Contradiction checks themselves are limited. They only cover the fact types and wording the parser knows how to compare.

Follow one sentence into fields

Use this fictional classroom sentence:

```
A fictional 45-year-old person has a dry cough for three days.
Temperature is 38.5 degrees Celsius.
```

The parser looks for separate patterns, not one magical “understanding” operation. It can match a number with “year-old” for age, a supported cough alias, a duration amount and unit, and a labeled temperature with a unit.

What the text says	Possible structured field
45-year-old	age_years = 45
cough	symptom name = cough; not negated
three days	duration amount = 3; unit = days
38.5 degrees Celsius	temperature value = 38.5; source text retained

This walkthrough illustrates the types of facts the parser is designed to extract. Exact parsing depends on the specific sentence structure and patterns in the code.

Fields not stated—such as heart rate—remain missing. The parser does not invent them.

Input safety checks

The safety module looks for a finite set of phrases associated with personal health, direct care requests, and selected urgent language. If a boundary is triggered, the agent does not continue its educational evaluation.

Why stop? The demo is intended for fictional exercises, not real requests for care. A fixed boundary message can direct users away from treating the program as a medical service.

The web server also requires the word “fictional” or “synthetic” in Reasoning Agent input. That requirement states the intended scope. It cannot verify the truthfulness of a user's statement.

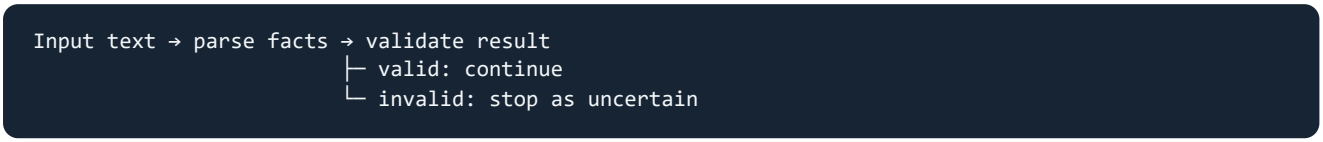
> Phrase matching can miss unexpected expressions or react to a phrase in a context it did not anticipate. It is not an emergency detector or a clinically validated safety mechanism.

Safety checks are a gate: the software either continues or returns a boundary response. They are not a score saying how risky a case is.

Validation: check the shape

Validation asks whether data meets required rules. It can check that text is not empty, does not exceed a length limit, and produced at least one supported fact. It can also list ambiguity, unsupported wording, and contradictions.

Why validate separately from parsing? Parsing tries to find facts. Validation decides whether the extracted result is acceptable for the next step. Separating these jobs makes failures easier to explain and test.



Validation checks form and consistency under explicit rules; it does not establish that a statement is true. A correctly formatted fictional measurement remains a fictional input.

If a validation gate fails, MEDNEX returns issues rather than trying to make an impressive-looking answer from weak data.

What is a knowledge graph?

A graph is a set of nodes connected by edges. A node is a named item; an edge is a named relationship between two items. MEDNEX uses a directed graph: each edge has a source and a destination.

```
[symptom: cough] ——illustrative link——> [synthetic pattern alpha]
```

The graph file contains synthetic symptom, observation, candidate-label, question, investigation-concept, and data-quality nodes. When a fact is extracted, the graph engine looks for a matching node, follows certain outgoing links, and records which links it visited.

Why a graph? It demonstrates connected information and makes it possible to return an explanation trail: “this node was visited because this extracted fact matched it.”

> A graph link is not proof. In MEDNEX, graph associations are explicitly synthetic and are not clinical evidence, a ranking, or a probability.

The graph also validates that edge types connect the expected categories, such as symptom to candidate label. This checks data consistency at load time.

Graph walking, step by step

1. Read the extracted facts, such as “cough present.”
2. Convert a fact name to the graph's node identifier.
3. If the node exists, inspect its outgoing edges.
4. Record each edge and why it was considered.
5. Collect connected synthetic candidate nodes.
6. Report missing nodes rather than silently making them up.

This is often called graph traversal or graph investigation. The engine follows only the relationships configured in the data. It does not search the internet or learn new relationships while running.

Negation changes how evidence is described. A negated symptom is treated as contextual in selected graph outputs rather than as positive support. That is an explicit software rule.

Why record visited edges? So an educational report can show its route. Transparency helps a student inspect the dataset and detect surprising connections.

Class exercise: Draw three circles for two symptom nodes and one synthetic candidate, then draw arrows matching the bundled graph. Ask which arrows are visited when one symptom is present.

If/then rules

A production rule is an instruction in the form “if these conditions match, then add a result.” MEDNEX's symbolic engine loads a few such synthetic rules from JSON.

> **IF condition A AND condition B are true**
> **THEN add a named evidence flag**

For example, one teaching rule says that if cough is explicitly present and fever is explicitly present, add a flag associated with one synthetic pattern. The output is a rule trace that records which conditions matched.

Why use symbolic rules? They are easy to inspect. A person can read the conditions and see what the program checked. They show how explicit logic differs from a learned model.

Each condition can be matched, missing, or not matched. “Missing” is important: the program did not receive that fact. It is not automatically a negative match.

> The example rules and signals are invented teaching content. They are not clinical rules and do not establish diagnoses.

Forward chaining

Forward chaining means repeatedly applying rules when new information becomes available. A first rule can create a flag; a later rule can use that flag as one of its conditions.

```
Round 1: cough + fever → create alpha_supported
Round 2: alpha_supported + headache → create beta_supported
Round 3: no new flags → stop
```

The engine tracks rules that have fired so it does not fire the same rule again and again. It repeats while at least one new flag is added. When a complete pass adds nothing, the process is finished.

Why do this? It demonstrates how small explicit rules can build on other explicit results. It also creates a trace that can explain the chain.

A second rule may create conflicting evidence for the same synthetic signal. The engine stores supporting and conflicting rules separately rather than deleting one side.

> **Important:** Forward chaining is a method of applying written rules. The name does not make the rule set medically knowledgeable.

The agent session

An agent session stores the fictional case, follow-up answers, question count, previous synthetic outputs, and other state. Think of it as a folder that keeps the conversation's educational context between turns.

When a question is answered, the server retrieves the session, records the answer under the current question, and runs the agent cycle again. The session is held in local memory, not written to a permanent database. Old sessions are capped.

Why keep state? Without it, the program would not know which question was answered or which questions were already asked. State allows the next turn to build on the prior turn.

The agent closes sessions after completion or a safety stop. Trying to answer a session with no pending question is rejected rather than silently changing the case.

> This session mechanism is a classroom prototype. It is not a secure medical record or production account system.

Follow-up answers are interpreted narrowly

The agent can map simple answer phrases into states such as present, absent, reported, not reported, or unknown. It uses explicit lists of positive, negative, and uncertain words.

For a symptom question, “yes” may map to present; “no” may map to absent; “not sure” maps to unknown. For a measurement question, “no” may mean “measurement not reported,” which is different from saying the measurement was zero.

Why have an unknown state? Real text can be unclear. Forcing every answer into yes or no would create false certainty. An unknown answer can be preserved and may lead to an uncertain ending.

The system uses a limited pattern of words. A complicated answer can be classified as unknown, even if a human could interpret it. This is another example of deterministic but limited language handling.

One agent cycle

```
1. Check safety.
2. Parse the fictional text.
3. Add follow-up answers to the extracted state.
4. Validate the facts.
5. Run graph, symbolic, Bayesian, and eligible ML steps.
6. Rank unasked questions.
7. Let the planner choose a state.
8. Build an explainability report.
9. Build a fixed user-facing answer.
```

The order matters. Safety comes before educational reasoning. Validation prevents some unsuitable input from entering later components. The planner uses the results to ask one question, stop uncertain, or finish.

Why rerun after an answer? Because a new fact can change the synthetic calculations and which questions remain unasked. The agent calculates a new turn from the updated state.

The web answer is built from the turn's actual extracted symptoms and curated guidance. It does not summarize every internal computation. This protects the intended simple interface but also means the response is not a technical report.

The planner's decision order

The planner is a set of gates, not a prediction model. It checks safety, validity, contradictions, ambiguity, available questions, and the follow-up limit.

Condition	Planner result
Safety boundary is active	Stop at professional-assessment boundary
Input is invalid or facts conflict	Stop as uncertain
A useful modeled question remains	Ask the top-ranked question, within the limit
Question limit reached with questions remaining	Stop as uncertain
No positive-gain questions remain and outputs exist	Complete the educational calculation

Why a decision order? If several conditions apply, the program needs a predictable rule for which one takes priority. Safety and data quality must be checked before asking another question.

The planner does not choose which synthetic label is “the answer.” Even at completion, it says uncertainty remains. Its output describes the next software action, not a patient recommendation.

What is probability?

Probability is a number used to describe how much weight a calculation assigns to an outcome. It ranges from 0 to 1. Multiply by 100 to write it as a percentage.

> **0.25 × 100 = 25%**

If there are three outcomes that cover all possibilities, their probabilities must add to 1. For example, 0.4 + 0.35 + 0.25 = 1. The values can be read as 40%, 35%, and 25%, which total 100%.

Why does the total matter? If the possibilities are meant to be a complete set, anything less than 100% leaves unexplained weight, and anything more than 100% counts too much weight. MEDNEX validates that priors sum to 1 and that posterior values sum to 1.

That is only a math rule. It does not prove that the possibilities are complete or that the numbers are true in real life.

Prior: the starting value

A prior is the starting probability assigned to a candidate before adding the evidence used in a calculation. Imagine a board with 100 tokens: 40 tokens labeled Alpha, 35 Beta, and 25 Other. Those counts represent the starting values 0.40, 0.35, and 0.25.

In a real scientific analysis, priors need an appropriate evidence basis. In MEDNEX, these are explicitly invented teaching values. Their purpose is to make the arithmetic visible.

Why call it a prior? It comes before the modeled evidence update. It does not mean “a doctor's first opinion,” and it should not be confused with a real-world chance for a person.

> **Plain-language phrase:** “The prior is where this toy calculation starts.”

The three values in the data file add to 1. This lets the code treat them as a complete starting distribution for its artificial labels.

Likelihood: a teaching lookup

A likelihood is another configured number. It answers a narrow mathematical question: “For this synthetic label, what number does our teaching table assign to this evidence?”

For example, the file assigns 0.80 as the likelihood of “cough present” for `synthetic_pattern_alpha`. This does not mean that cough occurs in 80% of real people in any health category. It means the table contains the number 0.80 for this artificial exercise.

Why use likelihoods? They let the program show how evidence can adjust starting weights. Without them, the Bayesian calculation has nothing to multiply by.

Each evidence key is explicit. Cough-present and cough-absent have different entries. A missing symptom is not automatically treated as an absent symptom. Only configured, recognized evidence keys participate.

> A likelihood table can be perfectly valid JSON and still have no real-world evidential basis. The MEDNEX file says these values are non-empirical and synthetic.

Multiply to get a temporary weight

The engine begins with one prior and multiplies by the likelihood for each evidence item. Multiplication makes a new temporary weight. It is called unnormalized because all the candidate weights do not necessarily add to 1 yet.

> **Weight = Prior × Likelihood 1 × Likelihood 2**

With the example values for Alpha, the computation is: 0.40 × 0.80 = 0.32; then 0.32 × 0.75 = 0.24. The temporary weight is 0.24.

Multiplication is the compact way the code applies all selected likelihood values in sequence. It is not the program “counting symptoms.” The values themselves come from the JSON teaching table.

Each label gets its own multiplication because each label has its own prior and its own likelihood values.

Check your understanding: If a starting value is 0.5 and a likelihood is 0.2, the product is 0.1. That is 10 hundredths. There is no need to interpret it as a real medical chance.

Normalize: make the weights add to one

Suppose the temporary weights for Alpha, Beta, and Other are 0.24, 0.02625, and 0.015. Their total is 0.28125. They do not add to 1, so we divide each weight by the total.

> **Updated share = That label's weight ÷ Sum of all labels' weights**

For Alpha, 0.24 ÷ 0.28125 = 0.8533 approximately. Repeat for Beta and Other. Once divided, the three values add to about 1. Tiny differences can appear because decimals are rounded for display.

Think of the temporary weights as portions of a pie whose total size is 0.28125. Dividing by the total tells us what fraction of the full pie belongs to each label. The normalized shares sum to 100%.

Why normalize? It makes a complete probability distribution that can be checked and used by later calculations such as entropy.

Full worked Bayes example

Use the actual synthetic prior and likelihood entries from the bundled file. Imagine the only evidence keys in this small worked example are cough-present and fever-present.

Label	Prior	Cough	Fever	Multiply
Alpha	0.40	0.80	0.75	0.24000
Beta	0.35	0.30	0.25	0.02625
Other	0.25	0.20	0.30	0.01500

Add the weights: 0.24000 + 0.02625 + 0.01500 = 0.28125. Divide each weight by that sum:

Label	Division	Result
Alpha	0.24000 ÷ 0.28125	0.8533
Beta	0.02625 ÷ 0.28125	0.0933
Other	0.01500 ÷ 0.28125	0.0533

The result adds to about 1. This is an example of arithmetic, not evidence about real health. A full input may contain additional configured observations; the agent calculation uses only eligible recognized evidence.

Why the independence assumption matters

To multiply likelihoods for multiple evidence items, the toy Bayesian calculation treats those items as conditionally independent given a candidate. In plain language, it pretends that once a label is fixed, each evidence item can be handled separately.

This is a simplifying assumption, not a fact established by MEDNEX. Real-world measurements and symptoms may be related. If related evidence is multiplied as if independent, the calculation can count overlapping information more than once.

Why use the assumption in a teaching demo? It gives students a tractable example of a common probability method. The important lesson is to state assumptions instead of hiding them.

Plain-language summary: The multiplication is convenient, but it depends on an assumption. If the assumption is wrong, the computed values may not represent the intended situation.

The project explicitly separates Bayesian output from machine-learning output. It does not average their numbers together.

Missing is not absent

Suppose a fictional sentence never mentions headache. The parser has no headache fact. It should record “not reported,” not “headache absent.” The person may have it, may not have it, or the sentence may simply omit it.

MEDNEX's Bayesian evidence extraction treats unmentioned observations as missing. Its ML feature representation also has a distinct `not_reported` state. This is a key data-quality principle.

Why does the difference matter mathematically? “Absent” is an explicit evidence state with its own configured likelihood. “Not reported” generally contributes no such evidence key. Treating them as the same would insert information that the input did not provide.

> **No mention ≠ explicitly absent**

In ordinary language: silence is not a “no.” This principle applies in surveys, forms, databases, and many other programs —not just MEDNEX.

A row, a feature, and a label

A row is one example in a table. A feature is one input column. A label is the answer the training table gives for that row.

cough	fever	headache	label
present	present	absent	synthetic alpha
absent	not reported	present	synthetic beta

In this fictional table, cough, fever, shortness of breath, and headache are features. The label is one of the synthetic teaching classes. The classifier learns a mapping from feature combinations to labels.

Why use a table? Models need data in a consistent form. MEDNEX validates that each feature uses one of a small set of allowed state words and that labels are marked synthetic.

A training row is not automatically an authoritative example. A model can faithfully learn patterns in poor, biased, tiny, or invented data.

Training and testing

Training means giving a model examples so it can fit internal patterns. Testing means checking predictions on examples held aside from training. MEDNEX has 50 hand-authored rows and uses a 75%/25% split.

> **50 rows → 38 training rows + 12 test rows**

The test count is rounded up in the code. A fixed random seed makes the split repeatable. Stratification aims to keep class proportions similar in training and test groups.

Why hold examples out? If we test only on examples the model practiced on, the score can make memorization look like general success. A separate test group gives a more useful software exercise.

But a held-out score is only meaningful relative to the data. Here the data is tiny and artificial. A high score would describe this synthetic split, not performance on people or real conditions.

Three kinds of classifier

MEDNEX trains logistic regression, a decision tree, and a random forest. The names describe different mathematical methods for mapping features to labels.

- **Logistic regression:** learns weights and uses them to score possible classes. MEDNEX uses it by default for a prediction.
- **Decision tree:** follows a sequence of feature questions, like a flowchart, until it reaches a class.
- **Random forest:** builds many decision trees and combines them.

scikit-learn implements the training algorithms. MEDNEX prepares the feature dictionary, builds the pipelines, fits the models, and reports outputs. It does not implement all model mathematics from scratch.

Why compare several classifiers? It is a way to demonstrate how different methods can be trained on the same small table. Their predictions should remain clearly identified as outputs from separate models.

> The models learn from artificial rows. The probability values they return are not calibrated clinical probabilities.

Accuracy: total correct guesses

Accuracy asks: out of all the test examples, what fraction did the model classify correctly? “Fraction” means correct examples divided by total examples.

> **Accuracy = Correct test predictions ÷ Total test examples**

Suppose a model gets 9 of 12 test rows right. Then 9 ÷ 12 = 0.75, which is 75%. This made-up number is only a teaching example.

Why measure it? It gives a simple overall summary of the model's test guesses. Its weakness is that it can hide poor results for a smaller class. If most examples are from one class, a model can look accurate by mostly guessing that class.

That is why MEDNEX also reports precision, recall, F1, and a confusion matrix. Each gives another view of the same synthetic test predictions.

Precision: when the model says “this label”

For one label, precision asks: among all test rows the classifier predicted as that label, how many truly had that label?

> **Precision = Correct positive predictions ÷ All positive predictions**

“True positive” (TP) means the label was predicted and it was correct. “False positive” (FP) means the label was predicted but it was wrong.

> **Precision = TP ÷ (TP + FP)**

Imagine a model predicted “blue team” 10 times. Eight guesses were correct and two were wrong. Precision is 8 ÷ 10 = 0.8, or 80%.

Why is this useful? It describes the trustworthiness of a model's positive predictions for that label, within the test dataset. It does not tell us how many true blue-team rows the model missed; that is recall.

Recall: how many true examples were found?

Recall asks: among all test rows that really belong to a label, how many did the model correctly find?

“False negative” (FN) means a row truly belonged to the label, but the classifier predicted another label. So recall divides correct finds by all actual examples of that label.

> **Recall = TP ÷ (TP + FN)**

If there are 10 real blue-team examples and the model finds 8, recall is 8 ÷ 10 = 0.8, or 80%. It missed two, so FN = 2 and TP = 8.

Why is recall separate from precision? A model could make only a few predictions and get them mostly right (high precision), while missing many real examples (low recall). Looking at both gives a fuller picture.

Again, these are test-set bookkeeping terms. The MEDNEX dataset is artificial; a score does not describe clinical capability.

F1: combining precision and recall

F1 combines precision and recall in one score. The formula is a harmonic mean. For this beginner guide, think of it as a combined score that becomes low if either precision or recall is low.

> **F1 = 2 × Precision × Recall ÷ (Precision + Recall)**

If precision and recall are both 0.8: 2 × (0.8 × 0.8) ÷ (0.8 + 0.8) = 1.28 ÷ 1.6 = 0.8, or 80%.

Why use F1? It summarizes the balance between “how often positive guesses were correct” and “how many real positive examples were found.” It cannot replace reading the two separate values, because the combined number hides detail.

If both precision and recall are zero, the formula would divide by zero. MEDNEX asks scikit-learn to return zero for certain undefined metric cases so the evaluation can still be reported.

Macro average and confusion matrix

Macro averaging gives each class equal importance in the final average. First find a metric for each class. Then add those values and divide by the number of classes.

> **For 3 classes: Macro score = (Score A + Score B + Score C) ÷ 3**

Suppose the class scores are 0.9, 0.6, and 0.3. The macro average is (0.9 + 0.6 + 0.3) ÷ 3 = 0.6. Each class gets equal weight, even if the dataset has different counts for each class.

A confusion matrix is a grid. Rows show the actual class; columns show the predicted class. The diagonal cells are correct predictions. Cells outside the diagonal show which classes were confused.

Why use both? Macro scores summarize across classes; a confusion matrix reveals specific mix-ups. In MEDNEX these are teaching diagnostics for a synthetic split only.

What does uncertainty look like?

Imagine choosing one of two equally likely colored cards. You do not know which card you will get; each has a 50% chance. That is more uncertain than a situation where the first card has 100% and the second 0% in the distribution.

Entropy is a number that measures this spread. For a fair two-choice situation, entropy is 1 bit. For a fully decided one-choice situation, it is 0 bits. “Bit” is a unit for information, like a tiny measuring unit; it is not a health measure.

Why use entropy? The information engine needs a number to compare the current distribution with possible distributions after hypothetical answers. Entropy provides that comparison.

Entropy does not say why the distribution exists, whether its inputs are good, or whether any candidate is true. It only measures the distribution the program was given.

> **Entropy = a number describing how spread out the toy probabilities are**

This is the starting idea for the next lessons. We will first write the calculation, then explain each symbol, then see how the program compares uncertainty before and after a hypothetical answer.

Entropy's formula, in plain language

The information engine computes Shannon entropy. It uses this compact formula for a set of candidate probabilities:

$$H(X) = -\sum_x P(x) \log_2 P(x)$$

Read it slowly: for every possible label, take its probability, multiply it by the base-2 logarithm of that probability, add all those answers, and change the sign to positive. The computer does this arithmetic; you do not need to calculate logarithms by hand.

Part	Meaning
H(X)	The entropy, or spread/uncertainty, of the candidate set X
P(x)	The probability value for one candidate x
∑ over x	Repeat for each candidate and add the results
$\log_2$	A base-2 logarithm; the result is measured in bits

The minus sign is part of the formula because logarithms of values between 0 and 1 are negative. It makes the final entropy non-negative.

How to read the math symbols

Math notation is a shorthand. Here is how to translate the symbols in MEDNEX's entropy and information-gain formulas into ordinary words.

- **H** is the name of the entropy calculation.
- **X** is the whole set of possible candidate labels.
- **x** is one label from that set. A lowercase letter means “one at a time.”
- **P(x)** is the current probability assigned to that label.
- ∑ means “add these values,” repeating for all candidates or answers.
- **$\log_2$** means logarithm to base 2. In this context it converts probability spread into a unit called bits.

Python's entropy function skips a term whose probability is zero. This avoids trying to compute the logarithm of zero, which is not defined by the usual real-number rules.

You may see a subscript such as the small 2 in $\log_2$. It names the base of the logarithm; it is not multiplication. You may see a subscript x under a summation sign; it says which values to repeat the addition over.

Do not memorize the symbol first. Learn the sentence it represents. The formula says: “Measure each candidate's contribution, then add all contributions.”

A tiny entropy example

Imagine two equally possible outcomes, each with probability 0.5. A probability of 0.5 is one half, or 50%. The outcomes together add to 1.

$$H = -[0.5 \times \log_2(0.5) + 0.5 \times \log_2(0.5)]$$

For base 2, $\log_2(0.5)$ equals −1. So each bracket term is 0.5 × (−1), or −0.5. Add the two terms: −0.5 + −0.5 = −1. The minus sign outside changes −1 to positive 1.

$$H = -[-0.5 - 0.5] = 1 \text{ bit}$$

Now imagine one outcome has probability 1 and the other probability 0. The result is completely decided in this distribution, so entropy is 0 bits. The computer skips the zero-probability term.

Why show this? It gives a scale: a fair two-choice distribution has more uncertainty than a distribution that puts all its weight on one choice. Entropy describes only these numbers, not truth or importance.

What does a question's answer probability mean?

The information engine asks hypothetical, prewritten educational questions. It estimates the probability of each possible answer using the current candidate distribution and a table of invented answer likelihoods.

$$P(a) = \sum_x P(x) \times P(a \mid x)$$

Read this as: for each candidate, multiply its current probability by the likelihood of answer *a* for that candidate. Then add the products. The vertical bar means “given.” So P(a | x) means “the table's likelihood of answer a, given candidate x.”

Use the cough question. Current values are Alpha 0.40, Beta 0.35, Other 0.25. For the answer “present,” the table values are 0.80, 0.30, and 0.20.

$$P(\text{present}) = (0.40 \times 0.80) + (0.35 \times 0.30) + (0.25 \times 0.20)$$
$$= 0.320 + 0.105 + 0.050 = 0.475$$

So this toy table says the hypothetical answer “present” has probability 0.475 (47.5%). This number is not a real-world frequency or clinical estimate. It is calculated from invented teaching values.

Update the candidates after an answer

If we imagine receiving answer *a*, the program updates the candidate probabilities. It uses the same multiply-and-divide idea as the Bayesian section.

$$P(x \mid a) = [P(x) \times P(a \mid x)] \div P(a)$$

In words: multiply the old probability by the answer likelihood, then divide by the overall probability of that answer. Division normalizes the new values so they add to 1.

For cough-present, the temporary values were Alpha 0.320, Beta 0.105, and Other 0.050. Their sum was 0.475. Divide each value by 0.475:

Candidate	Calculation	Updated share
Alpha	0.320 ÷ 0.475	0.6737
Beta	0.105 ÷ 0.475	0.2211
Other	0.050 ÷ 0.475	0.1053

Check: 0.6737 + 0.2211 + 0.1053 is about 1.0001. The tiny extra 0.0001 is rounding; unrounded program values sum to 1.

Expected entropy and information gain

Each possible answer creates its own updated candidate distribution and entropy. The engine calculates a weighted average over those possible outcomes. “Expected” means a probability-weighted average, not a promise.

$$\text{Expected entropy} = \sum_a P(a) \times H(X \mid a)$$

In the cough example, “present” has probability 0.475 and entropy about 1.2071 bits. “Absent” has probability 0.525 and entropy about 1.4571 bits.

$$\text{Expected entropy} = (0.475 \times 1.2071) + (0.525 \times 1.4571)$$
$$\approx 1.3384 \text{ bits}$$

Information gain asks how much lower the expected future entropy is than the current entropy:

$$\text{Information gain} = \text{Current entropy} - \text{Expected entropy after the answer}$$

Here, current entropy is about 1.5589 bits. Expected future entropy is about 1.3384 bits, so gain is about 0.2205 bits. The question is ranked by this mathematical difference.

A larger number only means greater expected reduction under the synthetic tables. It does not mean greater medical value, safety, urgency, or appropriateness.

The whole information-gain example

Put the steps together for the bundled cough question. The starting probabilities are Alpha 0.40, Beta 0.35, and Other 0.25. The current entropy is 1.5589 bits.

1. Calculate the chance of each hypothetical answer: P(present) = 0.475 and P(absent) = 0.525.
2. For “present,” multiply each starting probability by its answer likelihood and divide by 0.475. Updated values: 0.6737, 0.2211, 0.1053.
3. For “absent,” repeat using likelihoods 0.20, 0.70, and 0.80, then divide by 0.525. Updated values: 0.1524, 0.4667, 0.3810.
4. Calculate entropy for each updated set: about 1.2071 bits for present and 1.4571 bits for absent.
5. Weight and add those entropies: 0.475 × 1.2071 + 0.525 × 1.4571 ≈ 1.3384 bits.
6. Subtract from the original: 1.5589 − 1.3384 ≈ 0.2205 bits.

What did we learn? The computer can compare the fixed questions using expected change in its own toy probability distribution. The number comes from the chosen table, not a patient.

From internal results to a user answer

The agent creates a structured turn containing its session, status, safety result, extraction, optional engine results, decision plan, report, and change explanation. The user-facing response builder receives this object and chooses what to say.

It checks safety first, then missing extraction, contradictory symptoms, ambiguity, unsupported phrases, and whether a matching guidance topic exists. If it can continue, it inserts symptom labels into fixed text and adds curated lists.

If the planner chose a follow-up, the builder appends that exact prewritten question. The builder does not turn probability values into narrative or ask an LLM to write prose.

> **Three layers:** the engine produces facts and calculations; the report organizes details; the response builder selects safe, plain-language content for the chat.

Why have both report and chat answer? A technical report is useful for inspection. A short chat response is easier to read. They have different audiences, so they show different details.

What is a test?

A software test is a small program that checks another program. It prepares an input, runs a function, and checks whether the result matches expectations.

```
Input: a supported fictional cough phrase
Check: the parser records cough
Check: a negated cough is not recorded as present
Check: contradictory facts produce an uncertain state
```

Why tests? They catch accidental changes. If a developer changes a pattern and breaks negation, a parser test can show the problem before a person demonstrates the app.

The project uses pytest. Run all tests with `python -m pytest`. Tests cover models, parser, general responses, web routes, graph, symbolic engine, Bayesian arithmetic, ML, information gain, agent behavior, and reports.

A passing test means the code met the test's expectations. It does not prove every behavior is correct or medically valid. Tests check software behavior, not clinical evidence.

How to read the project as a beginner

Start with the entry point, then follow one request. Do not try to read every file at once. Trace the names in the call path.

1. `app.py` starts the server.
2. `web_server.py` receives the route and validates the request shape.
3. `guidance.py` handles the general chat, or `agent.py` runs the agent.
4. `nlp/parser.py` extracts supported facts.
5. Engine files calculate their own structured outputs.
6. `user_response.py` builds the plain user-facing agent response.

When reading a function, note its inputs, its named variables, its conditions, and its returned value. Follow one variable at a time. Search for where a function is called if you want to understand the full path.

Practice question: Which module decides what the browser displays? The browser JavaScript displays text from the response, while Python constructs the content and JSON fields.

Final classroom demonstration

1. Start the local server with `python app.py`.
2. In General Information, ask: “Please give general information about cough.” Point out the fixed template and curated content.
3. Try an unfamiliar phrase and observe clarification instead of invented guidance.
4. Switch to Reasoning Agent and enter only a fictional case: “A fictional person reports a cough.”
5. Explain the extracted facts, graph, rules, probability tables, ML, and information gain as separate educational components.
6. If a follow-up is asked, answer within the fictional exercise and show that the agent reruns with the session state.
7. Try a contradiction and show that the result remains uncertain.
8. Run `python -m pytest` to show that behavior is checked.

At every step, say what the program actually did. Do not call synthetic labels diagnoses, synthetic probabilities risk estimates, or phrase matching a full understanding of language.

> This classroom demo is not for real symptom questions. If a real person needs help, use an appropriate qualified professional or local emergency service—not this educational software.

Glossary and quick check

Word	Beginner definition
Parser	Code that searches for known patterns in text and creates fields.
Template	Prewritten sentence structure filled with selected values.
Prior	Starting probability in a toy calculation.
Likelihood	A configured evidence value for a candidate label.
Posterior	A normalized value after the configured evidence update.
Entropy	A mathematical measure of how spread out a probability distribution is.
Information gain	Expected reduction in that mathematical uncertainty.
Test	Code that checks another piece of code against an expected result.

Check you understand

1. Does an unmentioned symptom mean it is absent? **No.**
2. Does a graph link prove a real-world relationship? **No.**
3. Does an 80% synthetic probability mean 80% clinical risk? **No.**
4. Who writes the chat grammar? **The developers, using templates and prepared text.**
5. What is the safest result when input conflicts? **Do not force a conclusion; preserve uncertainty.**

Final message: MEDNEX is a transparent software teaching example. Learn from its structure and arithmetic, while remembering that its synthetic data is not medical knowledge.